ЛАБОРАТОРНАЯ РАБОТА № 1

Тема: **РАБОТА С ЭЛЕМЕНТАМИ УПРАВЛЕНИЯ В ПРИЛОЖЕНИЯХ WINDOWS FORMS**

1 ЦЕЛЬ РАБОТЫ

Учиться:

- создавать проекты Windows Forms;
- выполнять отладку приложений Windows Forms;
- работать с основными элементами управления.

2 ОБЪЁМ ЛАБОРАТОРНОЙ РАБОТЫ И ПОРЯДОК ЕЁ ВЫПОЛНЕНИЯ

Данная лабораторная работа выполняется в течение двух часов. Выполнение работы идет одновременно с изучением теоретической части. Для закрепления материала необходимо выполнить упражнения для самостоятельной работы.

3 ТЕОРЕТИЧЕСКАЯ ЧАСТЬ. ТЕХНОЛОГИЯ ВЫПОЛНЕНИЯ РАБОТЫ

3.1 Общие сведения о Windows Forms

Windows Forms – интерфейс программирования приложений (API), отвечающий за графический интерфейс пользователя и являющийся частью Microsoft.NET Framework.

Классы из пространства имен System.Windows.Forms позволяют писать программы, похожие на привычные приложения Windows. Они позволяют использовать кнопки, списки, текстовые поля, меню, окна сообщений и множество других «элементов управления».

Элементы управления – это то, что вы помещаете в форму. Они нужны для вывода информации, например, текстовой (элемент управления Label) или графической (элемент управления PictureBox), либо для выполнения определенных действий, например, выбора значения или перехода к другой форме после нажатия кнопки. Все элементы управления помещаются на форму.

В Windows Forms форма является видимой поверхностью, на которой отображается информация для пользователя. Обычно приложение Windows Forms строится путем помещения элементов управления на форму и написанием кода для реагирования на действия пользователя, такие как щелчки мыши или нажатия клавиш.

При выполнении пользователем какого-либо действия с формой или одним из ее элементов управления, создается событие. Приложение реагирует на эти события с помощью кода и обрабатывает события при их возникновении.

Windows Forms включает широкий набор элементов управления,

которые можно добавлять на формы. Перечислим некоторые из них:

– Label (Надпись).
– Button (Кнопка).
– ListBox (Список).
– CheckBox (Флажок).
– RadioButton (Переключатель).
– MessageBox (Окно сообщений).
– MenuStrip (Меню).
– TabControl (Управление вкладками).
– ToolStrip (Панель инструментов).
– TreeView (Дерево).
– DataGrid (Сетка данных).
– PictureBox (Изображение).
– RichTextBox (Текстовое поле с поддержкой формата RTF).

Если существующий элемент управления не удовлетворяет потребностям, в Windows Forms можно создать собственные пользовательские элементы управления с помощью класса UserControl.

В состав Windows Forms входят элементы пользовательского интерфейса с расширенными функциями, соответствующими возможностям мощных приложений, таких как Microsoft Office. Используя элементы управления ToolStrip и MenuStrip, можно создавать панели инструментов и меню, содержащие текст и рисунки, отображающие подменю и содержащие в себе другие элементы управления, такие как текстовые поля и поля с выпадающим списком.

С помощью конструктора Windows Forms Visual Studio, поддерживающего перетаскивание, можно легко создавать приложения Windows Forms: достаточно выделить элемент управления курсором и поместить его на нужное место на форме. Конструктор предоставляет такие средства, как линии сетки и «привязка линий» для преодоления трудностей выравнивания элементов управления. И в случае использования Visual Studio или компиляции из командной строки можно использовать элементы управления FlowLayoutPanel, TableLayoutPanel и SplitContainer для создания продвинутых разметок формы за минимальное время и с минимальными усилиями.

Если необходимо создать свои собственные элементы пользовательского интерфейса, пространство имен System.Drawing содержит широкий

набор классов, необходимых для визуализации линий, кругов и других фигур непосредственно на форме.

3.2 Создание простейшей формы и изменение еѐ свойств

Форма, как и каждый элемент управления, имеет набор свойств, которые позволяют настраивать внешний вид формы и еѐ расположение на экране.

1. Запустите программу Ms Visual Studio.
2. Выполните команду Файл – Создать – Проект. В появившемся диалоговом окне Создать проект в списке приложений выберите Приложение Windows Forms, введите имя проекта **Form1_1** и укажите место его сохранения. В результате на экране появится окно проекта **Form1_1**. В рабочей области находится форма в режиме конструктора (рисунок.1).

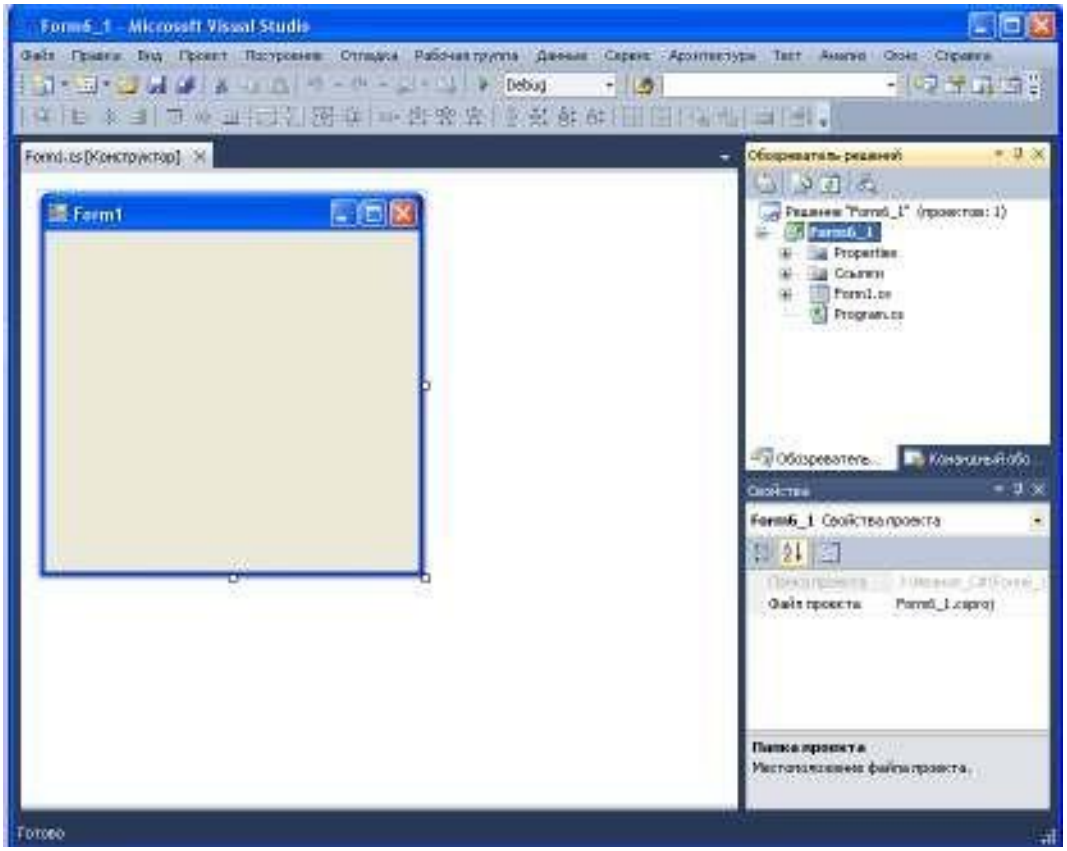

Рисунок 1 – Форма в режиме конструктора

3. Измените заголовок формы **Form1** на текст **Калькулятор**. Для этого выполните щелчок правой кнопкой мышки на форме в режиме конструктора и в контекстном меню выберите команду Свойства. В правом нижнем углу экрана появится окно Свойства. Измените значение свойства Text с **Form1** на **Калькулятор** (рисунок 2).

Рисунок 2 – Окно Свойства в режиме редактирования свойств формы **Form1**

4. С помощью свойства Name измените имя формы на **MainForm**.
5. В окне Свойства измените значения свойства Size: ширина – 470, длина – 500.
6. Установите расположение формы после загрузки в центре экрана с помощью значения CenterOwner свойства StartPosition.

Свойство StartPosition – расположение формы при загрузке.

Значения:

– Manual – положение формы на экране, задается свойствами Left и Top.

– CenterScreen – в центре экрана.

– WindowsDefaultLocation – положение задается системой, исходя из количества открытых окон и расположения их на экране.

– WindowsDefaultBounds – форма отображается в месте и с размерами окна, заданными по умолчанию в Windows.

– CenterParent – расположение формы в центре родительской.

7. Измените цвет фона формы на белый с помощью свойства BackColor.
8. Сохраните изменения и закройте решение.

3.3 Добавление на форму элементов управления и выполнение расчетов

1. Откройте проект **Form1_1**.
2. Откройте панель элементов и разместите на форме элементы управления и укажите их идентификаторы (свойство Name) согласно рисунку 3.

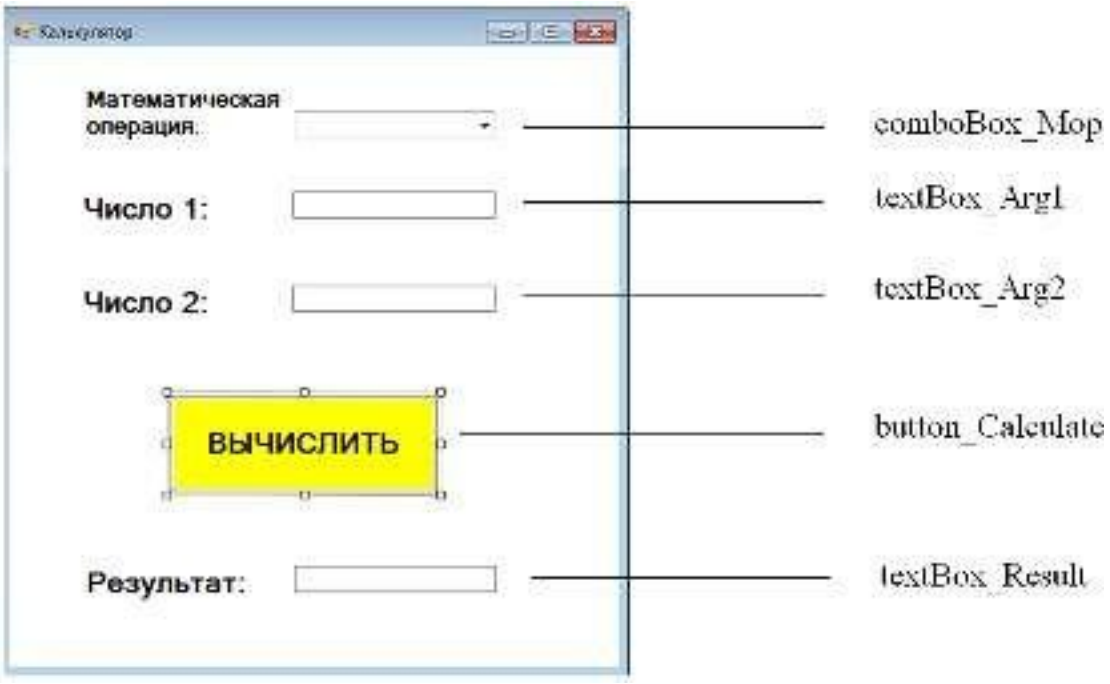

Рисунок 3 – Расположение элементов управления на форме к упражнению 2

3. Добавьте обработчик событий загрузки формы. Для этого щелкните по форме и в окне Свойства переключитесь на вкладку работы с событиями элементов управления, нажав на кнопку События , и дважды кликните по событию Load – справа появится надпись **MainForm_Load** (рисунок4), и в код класса **MainForm** будет добавлен метод-обработчик события загрузки формы с тем же именем (рисунок 5).

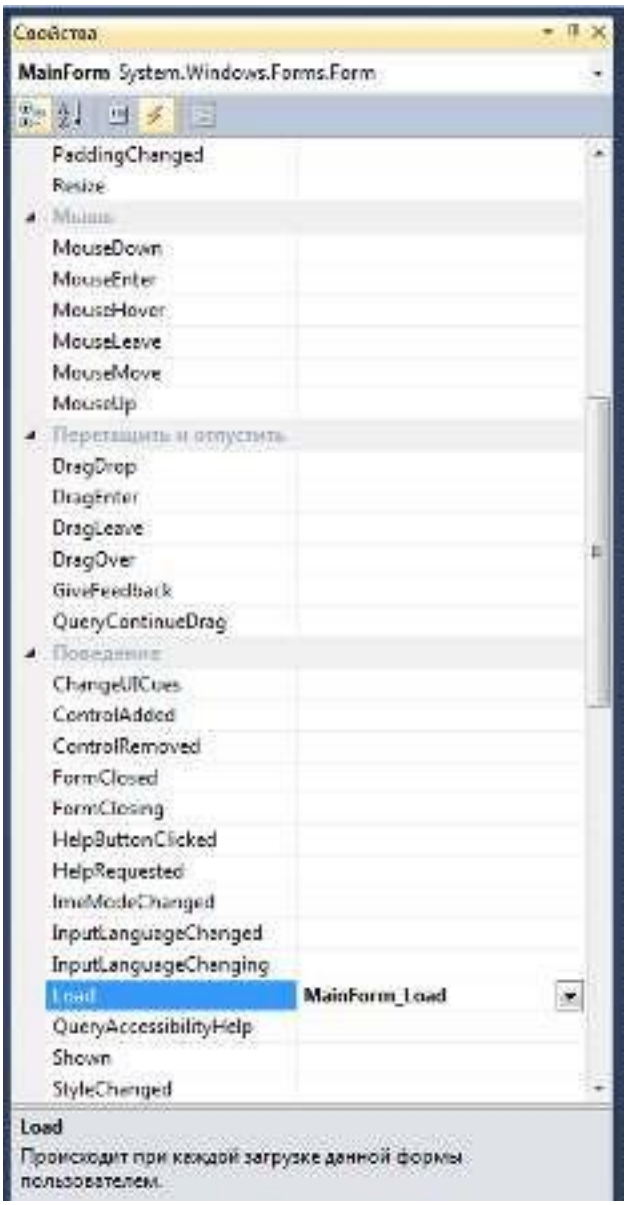

Рисунок 4 – Окно Свойства в режиме редактирования событий класса **MainForm** с добавленным обработчиком события Load – методом MainForm_Load

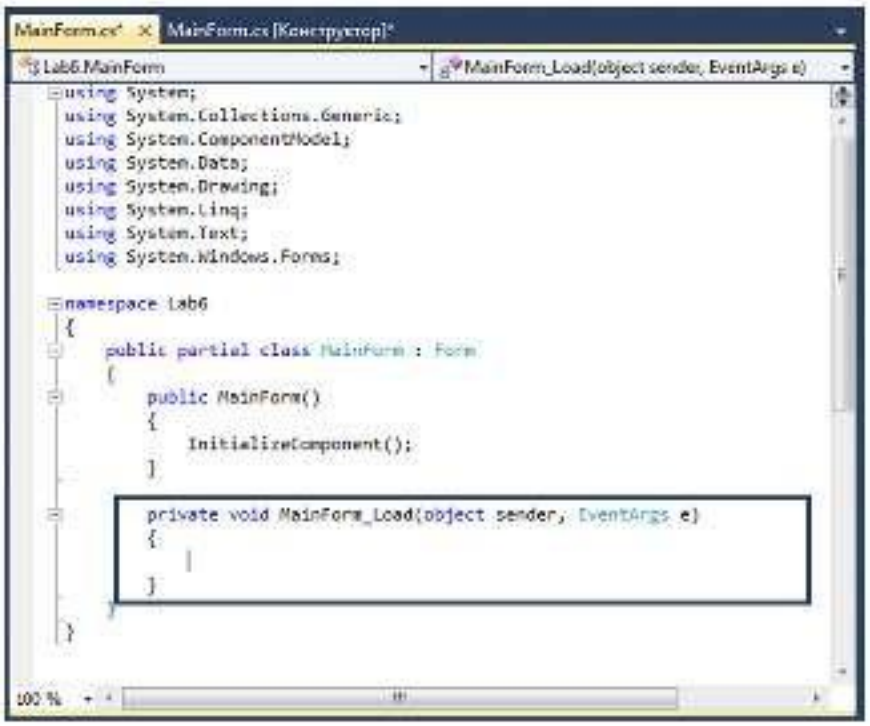

Рисунок 5 – Окно редактирования кода класса **MainForm** с добавленным обработчиком события Load – методом MainForm_Load

4. Отредактируйте метод MainForm_Load в соответствии с приведенным ниже кодом

```
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Drawing;
using System.Linq;
using System.Text;
using System.Windows.Forms;

namespace Form1_1
{
   public partial class MainForm : Form
   {
      public MainForm()
      {
         InitializeComponent();
      }

      private void MainForm_Load(object sender, EventArgs e)
      {
        // Добавляем элементы в список математических операций
        comboBox1_Mop.Items.Add("+");
        comboBox1_Mop.Items.Add("-");
```

```
        comboBox1_Mop.Items.Add("*");
        comboBox1_Mop.Items.Add("/");

        // Устанавливаем выбранный элемент с индексом 0
        comboBox1_Mop.SelectedIndex = 0;
      }
    }
}
```

5. Запустите проект на выполнение и убедитесь в заполнении списка математических операций.

6. Дважды щелкните по кнопке Вычислить. Автоматически будет добавлен обработчик события по умолчанию для данного элемента управления (для кнопки это щелчок) – метод button_Calculate_Click. Добавьте в него следующий код:

```
private void button_Calculate_Click(object sender, EventArgs e)
        {
          // Определяем текст, введенный пользователем
          // в поле "Число 1"
          string sx1 = textBox_Arg1.Text;

          // Преобразуем текст в число
          int x1 = Convert.ToInt32(sx1);

          // Определяем текст, введенный пользователем
          // в поле "Число 2"
          string sx2 = textBox_Arg2.Text;

          // Преобразуем текст в число
          int x2 = Convert.ToInt32(sx2);

         // Определяем, какую математическую операцию выбрал пользователь
          string mop = comboBox1_Mop.SelectedItem.ToString();

          double result = 0;

          if (mop == "+")
          {
             result = x1 + x2;
          }
          else if (mop == "-")
          {
             result = x1 - x2;
          }
          else if (mop == "*")
          {
             result = x1 * x2;
          }
          else
          {
             result = x1 / x2;
          }
```

```
        // Выводим результат в текстовое поле
        textBox_Result.Text = " = " + result;
    }
```

7. Запустите программу на выполнение. Проверьте работу программы при выборе различных математических операций.
8. Запустите программу на выполнение. Выберите математическую операцию «деление». В качестве первого числа наберите 7, в качестве второго числа – 0. Нажмите кнопку Выполнить. На экране появится сообщение об ошибке (рисунок 6).

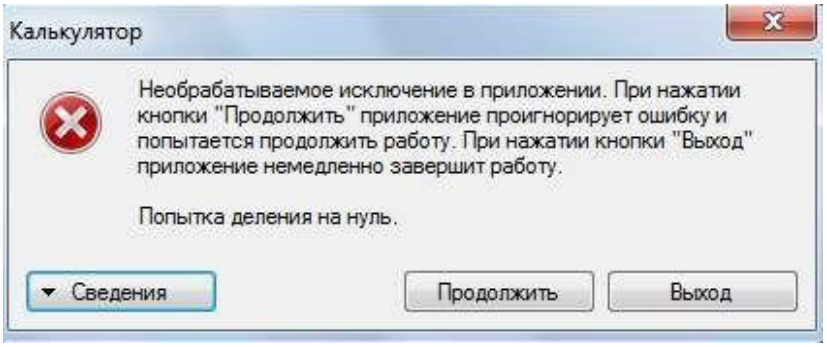

Рисунок 6 – Сообщение об ошибке «Деление на ноль»

9. Запустите программу на выполнение еще раз. Выберите любую математическую операцию. В качестве первого числа наберите 5, в качестве второго числа – любую букву. Нажмите кнопку Выполнить. На экране появится сообщение об ошибке (рисунок 7).

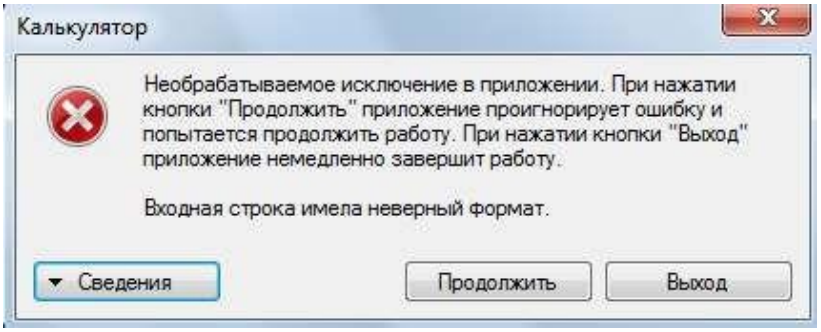

Рисунок 7 – Сообщение об ошибке «Неверный формат данных»

10. Сохраните решение и закройте программу.

3.4 Обработка исключительных ситуаций

Иногда при выполнении программы возникают ошибки, которые трудно предусмотреть или предвидеть. Например, при делении двух чисел необходимо предвидеть ситуацию деления на ноль (упражнение 2). Язык C# предо-

ставляет возможности для обработки таких ситуаций.

Основу обработки исключительных ситуаций в C# составляет пара ключевых слов try и catch. Эти ключевые слова действуют совместно и не могут быть использованы порознь. Ниже приведена общая форма определения блоков try/catch для обработки исключительных ситуаций:

```
try
{
   // Блок кода, проверяемый на наличие ошибок.
}

catch (ExcepType1 exOb)
{
   // Обработчик исключения типа ExcepType1.
}
catch (ExcepType2 exOb)
{
   // Обработчик исключения типа ExcepType2.
}
...
```

где ExcepType – это тип возникающей исключительной ситуации.

Когда исключение генерируется оператором try, оно перехватывается составляющим ему пару оператором catch, который затем обрабатывает это исключение. В зависимости от типа исключения выполняется и соответствующий оператор catch. Так, если типы генерируемого исключения и того, что указывается в операторе catch, совпадают, то выполняется именно этот оператор, а все остальные пропускаются.

Когда исключение перехватывается, переменная исключения exOb получает свое значение. На самом деле указывать переменную exOb необязательно. Так, ее необязательно указывать, если обработчику исключений не требуется доступ к объекту исключения, что бывает довольно часто. Для обработки исключения достаточно и его типа.

Следует, однако, иметь в виду, что если исключение не генерируется, то блок оператора try завершается как обычно, и все его операторы catch пропускаются. Выполнение программы возобновляется с первого оператора, следующего после завершающего оператора catch. Таким образом, оператор catch выполняется лишь в том случае, если генерируется исключение.

Исключения в C# представлены классами. Все классы исключений могут быть унаследованы от встроенного класса исключений Exception, который является частью пространства имен System. Именно поэтому все исключения представляют собой подклассы класса Exception.

Для исключения ошибок: деление на ноль и неправильный ввод дан- ных, добавим в программу «Калькулятор» обработку исключений.

1. Откройте проект **Form1_1**.
2. Отредактируйте код программы в соответствии с приведенным ниже кодом:

```
using System;
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Drawing;
using System.Linq;
using System.Text;
using System.Windows.Forms;
namespace Form1_1
{
   public partial class MainForm : Form
   {
      public MainForm()
      {
         InitializeComponent();
      }
      private void MainForm_Load(object sender, EventArgs e)
      {
         // Добавляем элементы в список математических операций
         comboBox1_Mop.Items.Add("+");
         comboBox1_Mop.Items.Add("-");
         comboBox1_Mop.Items.Add("*");
         comboBox1_Mop.Items.Add("/");

         // Устанавливаем выбранный элемент с индексом 0
         comboBox1_Mop.SelectedIndex = 0;
      }
      private void button_Calculate_Click(object sender, EventArgs e)
      {
        try
        {
            // Определяем текст, введенный пользователем
            // в поле "Число 1"
            string sx1 = textBox_Arg1.Text;
            // Преобразуем текст в число
            int x1 = Convert.ToInt32(sx1);
            // Определяем текст, введенный пользователем
            // в поле "Число 2"
            string sx2 = textBox_Arg2.Text;
            // Преобразуем текст в число
            int x2 = Convert.ToInt32(sx2);
```

```
            // Определяем, какую математическую операцию выбрал пользователь
            string mop = comboBox1_Mop.SelectedItem.ToString();
            double result = 0;
            if (mop == "+")
            {
               result = x1 + x2;
            }
            else if (mop == "-")
            {
               result = x1 - x2;
            }
            else if (mop == "*")
            {
               result = x1 * x2;
            }
            else
            {
               result = x1 / x2;
            }
            // Выводим результат в текстовое поле
            textBox_Result.Text = " = " + result;
         }
         // Обрабатываем исключение возникающее при делении на ноль
         catch (DivideByZeroException)
         {
            textBox_Result.Text = "Деление на 0!";
         }
         // Обрабатываем исключение при неккоректном вводе числа в консоль
         catch (FormatException)
         {
            textBox_Result.Text = "Это НЕ число!";
         }
         Console.ReadLine();
      }
   }
}
```

3. Запустите проект на выполнение и убедитесь в обработке исключений.
4. Сохраните решение и закройте программу.

4 УПРАЖНЕНИЯ ДЛЯ САМОСТОЯТЕЛЬНОЙ РАБОТЫ

1. В созданном ранее проекте Form1_1 на форму добавьте кнопку и соответствующий код для очистки текстовых полей.
2. Создать проект Windows Forms с именем Form6_sr. В проекте на форму добавить элементы управления для вычисления квадратного корня и степени числа и создать соответствующий программный код.

5 СОДЕРЖАНИЕ ОТЧЕТА

Отчет должен содержать листинги программ Form1_1 и Form1_sr, ис- ходные данные, результаты выполнения программ и ответы на контрольные вопросы.

6 КОНТРОЛЬНЫЕ ВОПРОСЫ

1. Что такое Windows Forms?
2. Как создать проект Windows Forms?
3. Элементы управления Windows Forms и их основные свойства.
4. Как изменить свойства элемента управления?
5. Как создать обработчик события элемента управления?
6. Какие ключевые слова используются для обработки исключений?
7. Приведите пример программного кода, содержащего обработку арифметического исключения «Деление на ноль».